

Raport tehnic: Personal Cloud Storage

Apetrei Ionuț-Teodor

Universitatea „Alexandru Ioan Cuza”, Iași
Facultatea de Informatică

1 Introducere

Viziunea pentru acest proiect este implementarea unei modalități sigure de stocare a fișierelor pe server de către utilizatori care să permită și accesarea de pe dispozitive diferite. Obiectivele proiectului includ crearea unui protocol de comunicare eficient, protejarea datelor prin criptare și redundanță cât și adăugarea unei interfațe grafice intuitive pentru client.

2 Tehnologii Aplicate

Pentru realizarea obiectivelor propuse am ales să folosesc protocolul TCP/IP care asigură că pachetele ajung la destinație, în ordinea corectă, ideal pentru transmiterea fișierelor fără a le corupe.

Comunicarea între server și clienți se va realiza în mod concurent, pe baza modelului pre-threaded folosind și multiplexare pe fiecare thread. Modelul este scalabil, capabil să servească un număr mare de clienți simultan.

Pentru transmiterea credențialelor la faza de autentificare se va folosi schimbul de chei ECDH, mai eficient decât RSA. Criptarea fișierelor: AES-256, algoritmul standard de criptare a datelor, folosit și de serviciile oferite de Google.

Pentru gestionarea bazei de date: fișiere XML, atât pentru datele de logare a clienților cât și pentru stocarea și servirea fișierelor și a metadatelor legate de acestea.

3 Structura Aplicației

Pentru ca aplicația să poată servi un număr cât mai mare de clienți vom folosi thread-uri pentru a putea beneficia de puterea computațională a tuturor nucleelor procesorului. Un singur client per thread e foarte risipitor (majoritatea timpului threadul va fi blocat așteptând să primească un mesaj), plus că de o parte thread-urile sunt mai ieftine decât procesele dar pe de alta și acestea vin cu un cost la creare și au un număr limitat, astfel încât am decis că voi folosi multiplexare per thread pentru a putea trata mai mulți clienți.

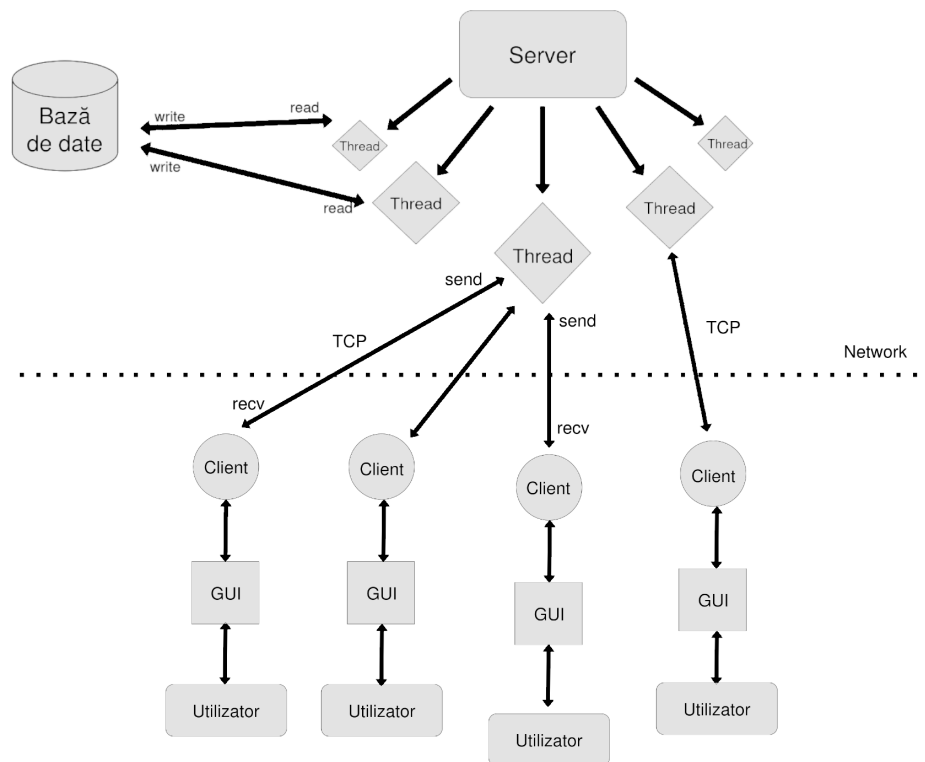


Fig. 1. Overview aplicație

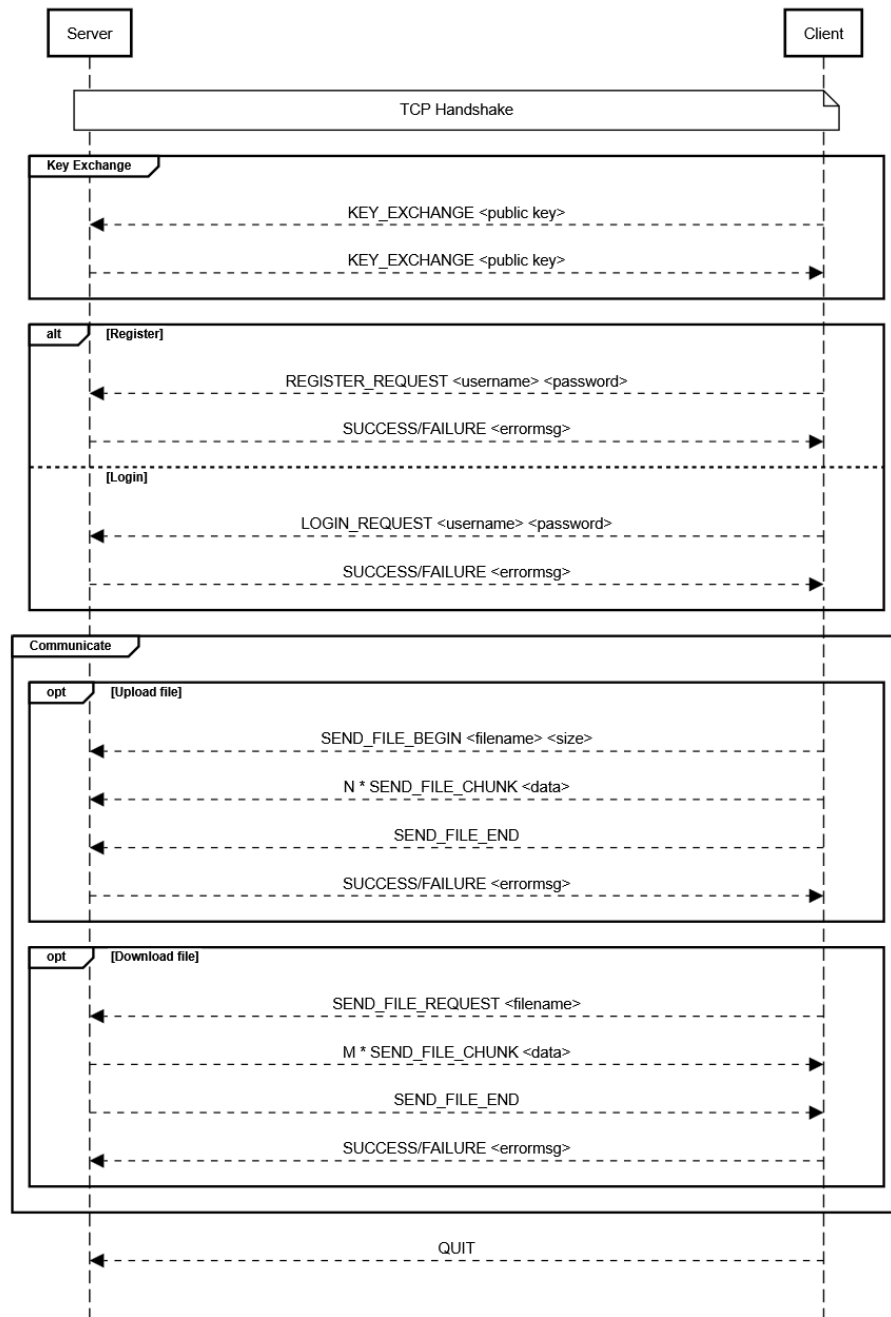


Fig. 2. Exemplu comunicare server-client

4 Protocolul de Comunicare

Pachetele transmise vor fi de forma:

- * int size: lungimea totală a pachetului (inclusiv size)
- * int flag: tipul pachetului
- * char* data: payload alocat dinamic (poate să lipsească)

Protocolul de comunicare va include următoarele tipuri de pachete ce pot fi transmise între server și clienți:

- QUIT
- KEY_EXCHANGE <public key>
- REGISTER_REQUEST <username> <password>
- LOGIN_REQUEST <username> <password>
- LOGOUT

Observație: Credențialele atașate la REGISTER_REQUEST și LOGIN_REQUEST vor fi transmise după ce se realizează KEY_EXCHANGE, astfel încât să poată fi criptate/decriptate prin cheia secretă corespunzătoare sesiunii curent în mod sigur.

- SEND_FILE_BEGIN <filename> <size>
- SEND_FILE_CHUNK <data>
- SEND_FILE_END
- SEND_FILE_REQUEST <filename>
- SUCCESS <optional>
- FAILURE <errmsg>

Alte operații pentru lucrul cu fișierele stocate pe baza de date de pe server:

- DIR_LIST_REQUEST
- CHANGE_CWD <dirname>
- CREATE_DIR <dirname>
- FILE_DELETE <filename>
- FILE_RENAME <filename> <newname>
- FILE_COPY <filename> <copypath>
- FILE_MOVE <filename> <moverpath>.

Observație: Ce este între <> reprezintă informația care trebuie să fie inclusă în payload pentru tipul de pachet respectiv (dacă este cazul).

5 Scenarii de Utilizare

Scenarii de succes:

- Utilizatorul se autentifică corect și încarcă fișiere criptate pe server. Acesta poate accesa fișierele de pe orice dispozitiv oricând chiar și după ce se încheie sesiunea curentă.
- Utilizatorul poate să se deconecteze de pe contul cu care s-a autentificat curent și să se logheze pe un alt cont, fără a se reseta conexiunea cu serverul.
- Mai mulți utilizatori sunt conectați simultan la server și execută operații cu fișiere, toate cererile vor fi tratate corect și rapid datorită concurenței.

Scenarii de eșec:

- Utilizatorul furnizează credențiale incorecte și serverul îi refuză accesul.
- Utilizatorul încearcă să se înregistreze cu un cont care există deja sau încearcă să se logheze cu un cont neexistent. Serverul îi dă un mesaj de eroare.
- Un fișier sau un director nu poate fi încărcat/creat deoarece are un nume invalid. Din nou serverul îi va trimite clientului un mesaj de eroare.

6 Concluzii

În mare, protocolul proiectat este eficient. Desigur că trebuie tratate cazurile de eroare ce pot apărea. Potențiale îmbunătățiri ar fi înlocuirea lui select de la curs, ori cu poll care nu are limitarea de 1024 de file descriptori, ori epoll, similar cu poll plus că e și mai eficient ($\mathcal{O}(1)$ în loc de $\mathcal{O}(n)$).

References

1. Alboaie Lenuța, Panu Andrei, „Rețele de calculatoare”, Facultatea de Informatică, Universitatea Alexandru Ioan Cuza, Iași, 2025
2. Differences between TCP and UDP <https://www.geeksforgeeks.org/computer-networks/diffie-hellman-key-exchange-and-perfect-forward-secrecy/>
3. Differences between select, poll and epoll, <https://www.hackingnote.com/en/versus/select-vs-poll-vs-epoll/>
4. Diffie-Hellman Key Exchange and Perfect Forward Secrecy, <https://www.geeksforgeeks.org/computer-networks/diffie-hellman-key-exchange-and-perfect-forward-secrecy/>
5. Secret Key Exchange (Diffie-Hellman) - Computerphile, <https://www.youtube.com/watch?v=NmM9HA2MQGI>
6. Elliptic Curves - Computerphile, <https://www.youtube.com/watch?v=NF1pwjL9-DE>
7. AES Explained (Advanced Encryption Standard) - Computerphile, <https://www.youtube.com/watch?v=O4xNJsjtN6E>